

	Air University Creative technology Department Spring 2026	
Full Stack Prog-Lab Final Term Project		
Subject:	Full Stack Programming Lab	
Class	BSSE-VI A	

Instructions:

- Create a GitHub account if you do not already have one.
- Create a repository named **Full-Stack-Programming-Lab**.
- Add me as a collaborator using the following email address:
sharifali.aulecturer@gmail.com
- Clone the repository to your local machine.
- Inside the repository, create a new folder named
Final_Term_Project_HLApp
- Push your completed work to GitHub.
- Prepare a Word or PDF document and include the following:
 - Screenshots of the output/results
 - Your GitHub repository URL
 - Upload the Word/PDF file to GCR for submission

Develop a full-stack web application that enables **patients** to book online **appointments** with doctors in a secure and structured **healthcare system**. The system allows administrators to manage doctors, patients, appointments, treatment records, and follow-up schedules efficiently. Once an appointment is confirmed and a doctor is assigned, the patient enters a continuous treatment cycle under medical supervision. The system supports treatment tracking, physical checkups, prescription management, and follow-up visits. Additionally, it provides automated notifications for appointments and medication schedules via email and mobile alerts.

The frontend must be developed using Next.js to ensure both client-side and server-side rendering for performance and SEO optimization. The backend should be built using Node.js and Express.js, with MongoDB used for persistent data storage. JWT-based authentication must be implemented to secure the system and restrict unauthorized access.

I.	Authentication System	(15 Marks)
	Design and implement a secure authentication system using JWT for role-based access control (Admin, Doctor, Patient).	
	Requirements: • User registration (name, email, password, role) • Password hashing using bcrypt • Secure login with JWT token generation • Protected routes for dashboard access • Role-based access control (Admin/Doctor/Patient) • Logout functionality (token/session removal)	

	• Prevent unauthorized API access	
II.	Doctor & Patient Management System	
	Develop a complete CRUD system for managing doctors and patients within the hospital system. Requirements: • Add, update, view, and delete doctors • Add, update, view, and delete patient records • Store all data in MongoDB using schema models • Assign doctors to patients • Validate all input fields Data Constraint: • Insert and maintain at least 15 doctor records and 15 patient records	
III.	Appointment & Treatment Management System	
	Implement a complete appointment booking and treatment lifecycle system. Requirements: • Book new appointments by patients • Approve or reject appointments by admin/doctor • Assign doctor to confirmed appointment • Track treatment progress after appointment confirmation • Maintain follow-up visit history • Store physical checkup records • Update treatment status after each visit	
IV.	Prescription & Medical Record System	
	Develop a module for managing prescriptions and medical history. Requirements: • Add prescriptions after consultation • Store medication details and dosage schedule • Maintain patient medical history • View previous treatment records • Link prescriptions with appointments	
V.	Notification System	
	Implement a notification system to improve patient engagement and treatment compliance. Requirements: • Appointment confirmation notifications • Medication reminders based on schedule • Follow-up visit reminders	

	- Email notifications - Mobile alert simulation (UI-based or API-based)	
VI.	Next.js Frontend Application	
	Develop a responsive and modern frontend using Next.js for the system. Requirements: - Login and registration pages - Role-based dashboards (Admin/Doctor/Patient) - Appointment booking interface - Treatment and prescription views - Protected routing system - Reusable components - API integration using Axios or Fetch	
VII.	Notification & UI Feedback System	
	Provide real-time feedback for user actions. Requirements: - Success messages for booking and updates - Error handling messages - Delete and update confirmations - Use toast or alert notifications	
VIII.	System Quality & Code Structure	
	The system will be evaluated based on code quality and UI design. Requirements: - Clean and responsive UI - Proper folder structure (frontend/backend separation) - Reusable components - Readable and maintainable code - Consistent naming conventions	

